



Process Matters

Architecture And Tech Stack

Version 1

Created by: Shelja, Simrat, and Vaneet

Last Updated by: Shelja, Simrat, and Vaneet

January 2, 2026

Covalience, LLC
2135 City Gate Lane
Suite 300
Naperville, Illinois 60563
United States of America



© Copyright 2018 Covalience™, LLC. All Rights Reserved.

Proprietary and Confidential

DOCUMENT CONFIDENTIALITY STATEMENT

The information in this document is proprietary and confidential and only to be used by the person(s) to whom it is addressed below for the purpose of engaging in business with Covalience, LLC. The information contained herein may not be shared or disclosed to any other person(s) outside of the company for which this document was prepared.

It may not be reproduced in whole or in part, nor may any of the information contained herein be disclosed without the prior consent of the directors of Covalience, LLC ('the Company').

Any form of reproduction, dissemination, copying, disclosure, modification, distribution, or publication of this material for any purpose other than to engage in business with Covalience and to any person not employed by the company to which this material is addressed is strictly prohibited.

Document Prepared for:

Process Matters

This document was prepared by:

Shelja and Aastha Tyagi

Covalience, LLC

2135 City Gate Lane, Suite 300

Naperville, IL 60563

+1 (630) 780-1090

betterdevteams@covalience.com

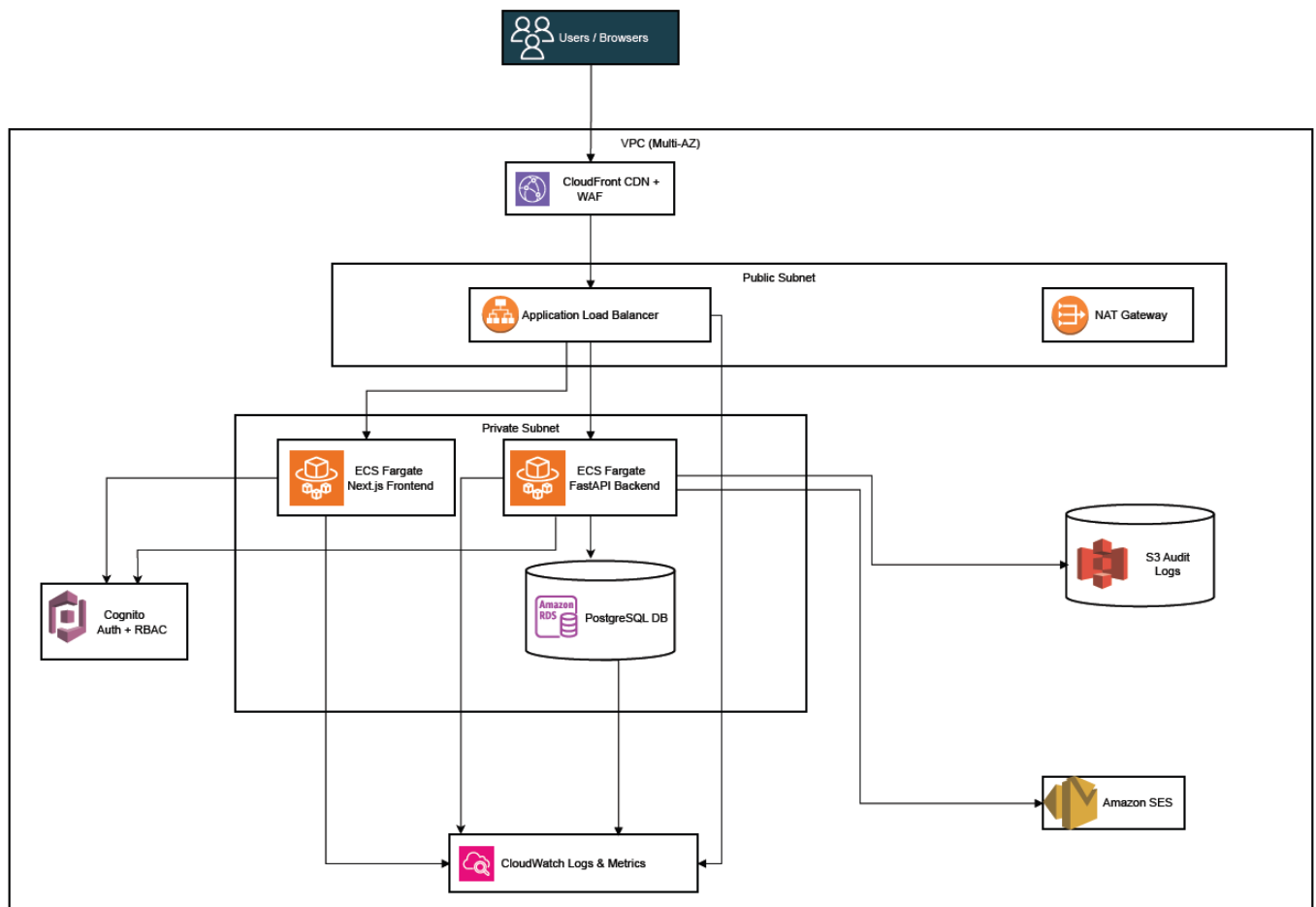
DOCUMENT HISTORY

Date	Author	Summary of Changes
12/26/2025	Shelja, Simrat, and Vaneet	Document Created

CONTENTS

Document Confidentiality Statement	2
Document History	3
1 Physical Architecture	5
1.1 Amazon CloudFront (Content Delivery Network)	5
1.2 AWS WAF (Web Application Firewall)	5
1.3 Application Load Balancer (ALB)	5
1.4 Amazon ECS (Containerized Application Layer)	6
1.5 Amazon Cognito (Authentication & Authorization)	6
1.6 Amazon RDS (Relational Database – Multi-AZ)	6
1.7 Amazon S3 (Object Storage)	6
1.8 Amazon SES (Email Service)	6
1.9 Networking & Security	6
1.10 Monitoring & Logging (Amazon CloudWatch)	6
1.11 Key Architecture Benefits	6
2 Logical Architecture	7
2.1 Frontend Layer (User Interaction)	7
2.2 Backend Layer (Security + Business Logic + Data)	8
2.2.1 Authentication & Authorization (Security Layer)	8
2.2.2 FastAPI Application (Main Backend API)	8
2.2.3 Database Layer (PostgreSQL + SQLAlchemy)	9
2.3 Webhooks + External Integrations	9
2.3.1 External API Access (Third-party systems calling you)	9
2.3.2 Webhooks (Your system notifying external systems)	9
2.4 Email Notifications (AWS SES)	9
2.5 Audit Logging & Error Monitoring	10
3 Why We're Choosing REST for This Application	10
4 Tech Stack Recommendations	12
4.1 FastAPI (Backend)	12
4.2 SQLAlchemy (ORM / Database Interaction Layer)	13
4.3 Next.js (Frontend)	13
4.4 PostgreSQL (Database)	13

1 PHYSICAL ARCHITECTURE



1.1 Amazon CloudFront (Content Delivery Network)

- Acts as the **primary entry point** for end users, serving content from edge locations globally.
- Reduces latency and improves performance by caching static and dynamic content.
- Integrated with **AWS WAF** to provide security at the edge.

1.2 AWS WAF (Web Application Firewall)

- Protects the application against common web threats such as SQL injection and XSS attacks.
- Applies security rules before traffic reaches backend services.
- Helps mitigate malicious and bot-driven traffic.

1.3 Application Load Balancer (ALB)

- Receives traffic forwarded from CloudFront.

- Distributes requests across multiple application instances for high availability.
- Supports SSL termination and health checks for backend services.

1.4 Amazon ECS (Containerized Application Layer)

- Runs containerized application workloads within **private subnets**.
- Enables horizontal scaling of services based on demand.
- Improves deployment consistency and operational efficiency.

1.5 Amazon Cognito (Authentication & Authorization)

- Provides user authentication and authorization for the application.
- Manages user sign-up, sign-in, password policies, and token issuance.
- Integrates with API Gateway and application services for secure access control.

1.6 Amazon RDS (Relational Database – Multi-AZ)

- Provides a managed relational database service with high availability and reliability.
- Automatically handles backups, patching, and failover.
- Deployed in private subnets to restrict direct internet access.

1.7 Amazon S3 (Object Storage)

- Stores static assets, application files, and backups.
- Highly durable and scalable storage service.
- Integrated with CloudFront for efficient content delivery.

1.8 Amazon SES (Email Service)

- Handles transactional and notification emails sent by the application.
- Ensures reliable and scalable email delivery.
- Reduces the need to manage in-house email infrastructure.

1.9 Networking & Security

- Deployed within a **VPC** with isolated public and private subnets.
- Security Groups and IAM roles enforce least-privilege access.
- Backend services remain inaccessible from the public internet.

1.10 Monitoring & Logging (Amazon CloudWatch)

- Collects logs and metrics from application and infrastructure components.
- Enables proactive monitoring and alerting.
- Supports operational visibility and troubleshooting.

1.11 Key Architecture Benefits

- **High Availability:** Multi-AZ deployment with load balancing.
- **Security:** Layered security using WAF, IAM, and private networking.

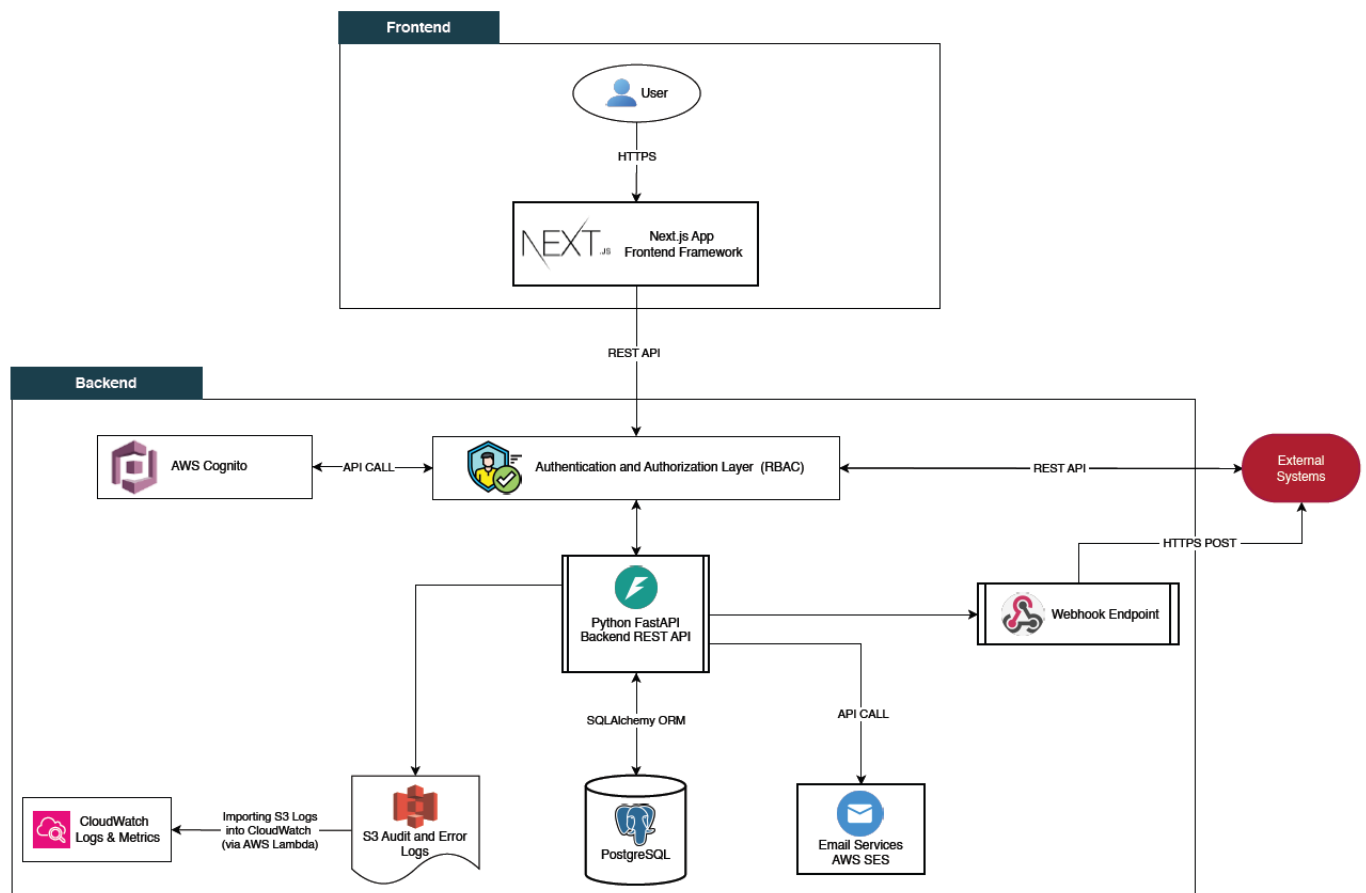
- **Scalability:** Automatic scaling of application services.
- **Performance:** Low-latency content delivery using CloudFront.
- **Operational Efficiency:** Fully managed AWS services reduce overhead.

2 LOGICAL ARCHITECTURE

This diagram illustrates the end-to-end operation of the website, from the user's initial opening to the system's authentication, request processing, data storage, and integration with third-party systems.

The architecture is divided into two main areas:

1. **Frontend** (what users see and interact with)
2. **Backend** (where business logic, security, database, and integrations happen)



2.1 Frontend Layer (User Interaction)

User → Next.js Web App

- A user accesses the website through their browser.

- The request is handled securely over HTTPS.
- The user interacts with the Next.js frontend application, which is responsible for:
 - Displaying pages/screens
 - Collecting user inputs
 - Sending requests to the backend APIs

Next.js → Backend via REST API

Whenever the user performs an action (such as logging in, creating data, viewing data, or updating something), the frontend sends a **REST API request** to the backend.

2.2 Backend Layer (Security + Business Logic + Data)

The backend is where all key application processing happens.

2.2.1 Authentication & Authorization (Security Layer)

AWS Cognito (Authentication)

- When the user logs in, AWS Cognito verifies their identity.
- Cognito returns secure authentication tokens (like a "digital access badge").

Auth + RBAC (Authorization)

- The backend has an **Authentication & Authorization Layer** that:
- Validates Cognito tokens
- Checks the user's role and permissions using **RBAC** (Role-Based Access Control)
- This ensures:
 - Only valid users can access APIs
 - Users can only perform actions their role allows

In short:

Cognito confirms who the user is, and RBAC confirms what they are allowed to do.

2.2.2 FastAPI Application (Main Backend API)

After the request is authenticated and authorized, it goes to the FastAPI backend, which:

- Receives REST API requests
- Applies business rules
- Performs system operations (create/update/delete)
- Triggers integrations (email, webhooks, external APIs)

- Writes data to the database

2.2.3 Database Layer (PostgreSQL + SQLAlchemy)

SQLAlchemy ORM

- The backend utilizes SQLAlchemy (Object-Relational Mapping, or ORM) to communicate with PostgreSQL.
- This provides structured and secure database access.

PostgreSQL

- PostgreSQL is the core database

2.3 Webhooks + External Integrations

This system supports two kinds of integration with external systems:

2.3.1 External API Access (Third-party systems calling you)

External Systems → REST API

Third-party systems (partners, external clients, etc.) can connect to the backend through an API interface.

- They send REST API requests to the backend
- Requests still go through authentication/authorization rules

This enables:

- Partner integrations
- Automated workflows
- System-to-system syncing

2.3.2 Webhooks (Your system notifying external systems)

Backend → Webhook Endpoint → External System (HTTPS POST)

Whenever important events occur (for example, an order is created, the status changes, or approval is complete), the backend triggers a webhook.

The webhook system:

- Sends an **HTTPS POST** request to external systems
- Allows real-time event-based notifications

This enables external systems to "listen" to your platform events.

2.4 Email Notifications (AWS SES)

Some system actions may require sending emails (password reset, alerts, confirmations).

The FastAPI backend calls AWS SES (Simple Email Service) to send emails reliably and securely.

2.5 Audit Logging & Error Monitoring

Security and reliability are built in through logging and monitoring:

S3 Audit and Error Logs

- Audit and error logs are stored in S3 for long-term retention and archival purposes.
- Audit logs record important actions like:
 - Who accessed what
 - What changes were made
 - When the action occurred

CloudWatch Logs & Metrics

- Logs stored in S3 are imported into CloudWatch.
- CloudWatch provides:
 - Centralized log viewing
 - Monitoring dashboards
 - Alerts when errors happen
 - Performance metrics tracking

This ensures observability and easy issue troubleshooting.

3 WHY WE'RE CHOOSING REST FOR THIS APPLICATION

We are choosing **REST APIs** for this application because REST is the most practical, scalable, and widely supported approach for systems that require **secure access, third-party integrations, webhooks, and strong operational monitoring**.

- **Best Fit for Third-Party Integrations (External Consumers)**

Since this platform must expose APIs to third-party systems, REST is the industry standard for external integrations.

Why it matters:

- Most third-party systems already have built-in REST support
- REST is easy for partners to understand and integrate quickly
- REST makes it easier to provide consistent documentation and SDKs

Result: **Faster partner onboarding + fewer integration issues**

- **Webhooks Naturally Work with REST**

Your system includes webhook support. Webhooks are, by design, REST-based HTTP calls (usually POST requests).

Why it matters:

- REST aligns directly with webhook patterns

- Standard HTTP status codes provide clear delivery signals (200/400/500, etc.).
- Easy retry and idempotency handling

Result: **Simpler and more reliable webhook implementation**

- **Simplifies Security Controls (Cognito + RBAC)**

With Cognito authentication and RBAC authorization, REST provides an immaculate and enforceable model.

Why it matters:

- Permissions can be enforced per endpoint
- Each endpoint maps to a business operation clearly
- Lower risk of accidental overexposure of data

Result: **Stronger security and easier access control management**

- **Better Logging, Auditing, and Monitoring**

REST endpoints are easier to monitor and audit because each endpoint corresponds to an explicit action.

Why it matters:

- Each endpoint has its own logs, latency stats, and error rates
- Easier to trace request paths in observability tools (CloudWatch, Datadog, etc.)
- Auditing becomes simpler ("User X called endpoint Y at time Z")

Result: **Higher operational clarity and easier compliance reporting**

- **Easier API Governance and Versioning**

REST provides straightforward API lifecycle management and versioning.

Why it matters:

- You can version APIs cleanly: /api/v1/...
- External clients can stay stable without breaking changes
- You can evolve the platform safely over time

Result: **Safer long-term evolution of the product**

- **Caching and Performance are Simpler**

REST integrates naturally with caching mechanisms (CDNs, API gateways, HTTP caching headers).

Why it matters:

- GET requests can be cached efficiently
- API Gateway + CDN caching is easy to configure

- Improves response times and reduces backend load

Result: **Better performance and scalability with less complexity**

- **Lower Development Complexity (Faster Delivery)**

REST has a smaller learning curve compared to GraphQL and is simpler to maintain over time.

Why it matters:

- Faster to implement and test
- Less overhead in schema, resolvers, and field-level auth
- Easier onboarding for new developers

Result: **Reduced engineering effort and lower long-term maintenance cost**

- **Strong Standardization and Tooling Support**

REST APIs have mature, widely adopted tooling and best practices.

Why it matters:

- Works seamlessly with OpenAPI / Swagger documentation
- Built-in support across platforms and languages
- Easier contract testing and API validation

Result: **High developer confidence + strong documentation standards**

4 TECH STACK RECOMMENDATIONS

We recommend using the following tech stack.

1. FastAPI for the backend
2. SQLAlchemy (ORM / Database Interaction Layer)
3. Next.js for the frontend
4. PostgreSQL as the database

This is a modern, highly reliable, and scalable technology stack that is widely adopted in production environments and well-suited for building secure, high-performance applications.

4.1 FastAPI (Backend)

FastAPI is a modern Python-based backend framework designed for building fast, secure APIs with excellent scalability.

Key benefits:

- **High performance:** FastAPI is one of the fastest Python frameworks, supporting asynchronous processing for efficient handling of multiple requests.

- **Secure and structured:** Built-in validation helps reduce errors and strengthens API reliability.
- **Faster development:** Automatic API documentation and clean architecture improve delivery speed and maintainability.
- **Easy integration:** Works seamlessly with third-party services, payment systems, authentication systems, and internal tools.

4.2 SQLAlchemy (ORM / Database Interaction Layer)

SQLAlchemy is the most widely used ORM in Python and is considered the industry standard for database interaction in production applications.

Key benefits:

- **Enterprise-grade and mature:** Highly stable and proven across large-scale applications.
- **Clean & maintainable data layer:** Separates business logic from database operations clearly.
- **Compatibility and flexibility:** Works exceptionally well with PostgreSQL and supports both synchronous and asynchronous workflows.
- **Migration support:** Works seamlessly with tools like Alembic for managing database migrations.

4.3 Next.js (Frontend)

Next.js is a widely used framework for building fast, responsive, and SEO-friendly web interfaces.

Key benefits:

- **Excellent user experience:** Optimized rendering results in faster page load times and smooth interactions.
- **SEO and performance-friendly: Supports server-side rendering, making it ideal for applications that require** visibility and speed.
- **Scalable frontend architecture:** Well-suited for future enhancements and modular UI development.
- **Enterprise adoption:** Trusted by leading tech companies and has strong long-term community support.

4.4 PostgreSQL (Database)

PostgreSQL is a production-grade relational database known for stability, data integrity, and flexibility.

Key benefits:

- **Highly reliable:** Strong transactional support ensures data consistency and integrity.

- **Scales effectively:** Suitable for applications that grow in data and user traffic.
- **Flexible data handling:** Supports structured relational data and advanced features like JSON storage.
- **Long-term support:** Proven technology with mature tooling and widespread cloud support.